# CSL7110: Machine Learning with Big Data

## Assignment 4

### Clustering, Web Search, and PageRank


**Name:** Dwivedi Jyoti Rajeshbhai
**Roll Number:** M25CSA010
**Course:** CSL7110
**Date:** April 7, 2026


**GitHub Repository:**
https://github.com/Jyoti-Dwivedi-010/ML_with_Big_Data_Assignment4

# 1 Introduction

This report contains the assignment having three independent but related components:

- **Part 1: Clustering** on the UCI Spam dataset using Farthest-First traversal for k-center and k-means++ initialization.
- **Part 2: Web Search** using an inverted index over a small webpage collection.
- **Part 3: PageRank on Spark** using iterative rank propagation on a directed graph.

All code was written in a Jupyter notebook and run on the provided datasets. The notebook was executed successfully, and the outputs are included as screenshots in the `images` folder.

# 2 Assumptions

The following assumptions were used while solving the assignment:

- For Part 1, the required values were taken as $k = 10$ and $k_1 = 50$.
- Randomness in k-means++ was made deterministic using a fixed seed.
- In Part 2, connector words were excluded from indexing but still counted in word positions, as required.
- Punctuation was replaced by spaces and all words were normalized to lowercase.
- Singular/plural normalization was applied only to the exhaustive mapping specified in the assignment.
- In Part 3, duplicate directed edges were removed using `distinct()` before rank computation.
- The supplied `small.txt` file in the current workspace parses to 100 nodes in the notebook output, since it is containing 100 unique nodes(IDs 1-100) and not 53 nodes. The report reflects the actual notebook output corresponding to these 100 nodes.

# 3 Part 1: Clustering

## 3.1 Problem Setting

The spam dataset contains 4601 points in 58-dimensional Euclidean space. Each point is represented as a Spark vector. It required the following functions:

- `readVectorsSeq(filename)`
- `kcenter(P, k)`
- `kmeansPP(P, k)`
- `kmeansObj(P, C)`

## 3.2 Methodology

**Read vectors.** The function `readVectorsSeq` reads each line of the spam dataset, splits the comma-separated values, converts them to floating-point numbers, and stores them as Spark vectors.

**Farthest-First traversal.** The `kcenter` function initializes with the first point and repeatedly adds the point farthest from the current set of centers. This is the standard farthest-first traversal for k-center clustering.

**k-means++ initialization.** The `kmeansPP` function selects centers probabilistically using the squared distance to the nearest existing center. The implementation uses a fixed random seed so results are reproducible.

**k-means objective.** The `kmeansObj` function computes the average squared distance of each point to its closest center:

$$\text{kmeansObj}(P, C) = \frac{1}{|P|} \sum_{p \in P} \min_{c \in C} \|p - c\|^2.$$

## 3.3 Results

The notebook output for Part 1 is shown below.

```
Part 1 Results
- Dataset points: 4601
- Dimension: 58
- kcenter runtime (sec): 0.294602
- kmeans++ objective: 26528.124826
- kcenter(k1) -> kmeans++ objective: 350760.434241
```

Figure 1: Part 1 clustering results on the spam dataset.

The observed results were:

- Dataset points: 4601
- Dimension: 58
- kcenter runtime: 0.294602 seconds
- k-means++ objective: 26528.124826
- k-center coreset followed by k-means++ objective: 350760.434241

### 3.4 Discussion

The k-center method is fast and provides a good set of diverse centers, while k-means++ is optimized for the k-means objective. The core experiment using k-center first and then k-means++ on the resulting centers demonstrates how a smaller representative set can be used to approximate the full dataset. The results confirm that the notebook implements all required clustering functions and produces the requested outputs.

## 4 Part 2: Web Search Using an Inverted Index

### 4.1 Problem Setting

The assignment required building a search engine over a small collection of webpages using an inverted index. The following classes and methods were implemented:

- `MySet`
- `Position`
- `WordEntry`
- `PageIndex`
- `PageEntry`
- `MyHashTable`
- `InvertedPageIndex`
- `SearchEngine`

### 4.2 Methodology

The implementation follows the assignment specification:

- All words are converted to lowercase.
- Connector words are ignored for indexing, but they still count in word positions.
- Punctuation marks are replaced by spaces.
- Singular and plural forms of the specified word pairs are normalized.
- The search queries are processed from `actions.txt`.

  The query types supported are:

- `addPage x`
- `queryFindPagesWhichContainWord x`
- `queryFindPositionsOfWordInAPage x y`

## 4.3 Results

The notebook output for Part 2 is shown below.

```
Part 2 Query Outputs
01. No webpage contains word delhi
02. stack_datastructure_wiki
03. stack_datastructure_wiki
04. Webpage stack_datastructure_wiki does not contain word magazines
05. No webpage contains word allain
06. stack_cprogramming
07. stack_cprogramming
08. stack_cprogramming
09. stack_oracle
10. stack_cprogramming, stack_datastructure_wiki, stackoverflow
11. stackmagazine

Matches answers.txt: True
```

Figure 2: Part 2 query outputs from the notebook.

The outputs match `answers.txt` exactly, as confirmed by the notebook:

`Matches answers.txt: True`

For comparison, the answer file is also shown in the workspace:

```
assignment4_solution.ipynb   answers.txt   demo.ipynb
Assignment 4- datasets > Q2- webSearch > answers.txt
No webpage contains word delhi
stack_datastructure_wiki
stack_datastructure_wiki
Webpage stack_datastructure_wiki does not contain word magazines
No webpage contains word allain
stack_cprogramming
stack_cprogramming
stack_cprogramming
stack_oracle
stack_cprogramming, stack_datastructure_wiki, stackoverflow
stackmagazine
```

Figure 3: Reference answers from `answers.txt`.

## 4.4 Discussion

The search engine correctly identifies pages containing query words and returns word positions within pages. The output strings follow the assignment's exact wording requirements, including the lowercase form `No webpage y found`. This is important for strict evaluation.

# 5 Part 3: PageRank on Spark

## 5.1 Problem Setting

The third task required implementing PageRank on a directed graph using Spark. The assignment specifies:

- Run for 40 iterations with $\beta = 0.8$.
- Remove duplicate directed edges.
- Treat the transition matrix $M$ as an RDD.
- Report the top 5 and bottom 5 nodes by PageRank score.

The PageRank update rule is:

$$r^{(t)} = \frac{1-\beta}{n} A + \beta M r^{(t-1)},$$

where $A$ is the all-ones vector scaled appropriately and $M$ is the transition matrix.

## 5.2 Methodology

The notebook implements the following steps:

1. Read the edge list from `small.txt` or `whole.txt`.
2. Parse each line into a directed edge $(u, v)$.
3. Remove duplicate edges using `distinct()`.
4. Build the adjacency structure and sparse transition matrix $M$ as an RDD.
5. Initialize all node ranks uniformly.
6. Iterate 40 times with damping factor $\beta = 0.8$.
7. Collect and report the top 5 and bottom 5 nodes.

## 5.3 Results

The notebook output for Part 3 is shown below.

```
PART 3: PageRank on Spark

Small Graph :
  Nodes (parsed): 100
  Note: current small.txt yields a different node count than 53.
  Top score: 0.035731
  Top 5 nodes:
    Node 53: 0.0357312
    Node 14: 0.03417091
    Node 40: 0.03363009
    Node 1: 0.03000598
    Node 27: 0.02972014

Whole Graph (1000 nodes, β=0.8, 40 iterations):
  Nodes: 1000

  Top 5 nodes (highest PageRank):
    Node 263: 0.00202029
    Node 537: 0.00194334
    Node 965: 0.00192545
    Node 243: 0.00185263
    Node 285: 0.00182737

  Bottom 5 nodes (lowest PageRank):
    Node 558: 0.0003286
    Node 93: 0.00035136
    Node 62: 0.00035315
    Node 424: 0.00035482
    Node 408: 0.0003878
```

Figure 4: Part 3 PageRank results on the small and whole graphs.

The notebook produced the following results:

**Small graph.**

- Parsed nodes: 100
- Top score: 0.035731
- Top 5 nodes: 53, 14, 40, 1, 27

**Whole graph.**

- Nodes: 1000
- Top 5 nodes: 263, 537, 965, 243, 285
- Bottom 5 nodes: 558, 93, 62, 424, 408

## 5.4 Discussion

The small-graph top score is very close to the expected value of approximately 0.036, which validates the implementation. The notebook also notes that the current `small.txt` file in the workspace parses to 100 nodes rather than 53, so the report records the observed result while acknowledging the dataset discrepancy.

The whole-graph experiment runs successfully and returns both the highest and lowest ranked nodes after 40 iterations. This satisfies the reporting requirements of the assignment.

# 6 Implementation Notes

A few implementation details are worth highlighting:

- Part 1 uses Spark vector objects and a compatibility helper for squared distance.
- Part 2 uses custom data structures to model an inverted index and search engine.
- Part 3 uses a strict RDD-based PageRank implementation, aligned with the assignment wording.
- Exact output strings were matched carefully for Part 2.

# GitHub Repository

The complete code notebook is available here:

<https://github.com/Jyoti-Dwivedi-010/ML_with_Big_Data_Assignment4>